# 60+ .NET Microservices Interview Q&A (Senior Level)

**Need Deeper Guidance?**
**If you want real 1-on-1 help:**
**Live mock interviews,Career mentorship,Resume reviews,LinkedIn profile optimization Connect with me:**

https://topmate.io/sandeeppal

---

## 🔹 1. What is Microservices Architecture? Explain with production scenario.

Answer:
Microservices architecture is a distributed system design style where an application is decomposed into small, independently deployable services, each aligned to a business capability.

### Key Engineering Properties:

- Autonomous deployment
- Decentralized data ownership
- Independent scalability
- Fault isolation
- Polyglot technology freedom

### Real Production Scenario:

Consider an Amazon-like E-commerce Platform:

| Service | Responsibility | Tech |
|---|---|---|
| Catalog Service | Product listing/search | .NET + MongoDB |

https://www.linkedin.com/in/sandeeppal/

| | | |
|---|---|---|
| Cart Service | User cart management | .NET + Redis |
| Order Service | Order lifecycle | .NET + SQL Server |
| Payment Service | Payment processing | .NET + Stripe |
| Notification Service | Emails/SMS | Node + SendGrid |

Each service:

- Has its own database
- Deploys independently
- Scales independently

If Cart Service fails, users can still browse products — fault isolation achieved

---

## 🔹 2. Microservices vs Monolith – When should you choose Microservices?

Answer:

### Monolith is better when:

- Team size < 5 developers
- Domain is simple
- MVP stage
- Deployment frequency is low

### Microservices is better when:

- System is domain-complex
- Teams are distributed
- Need independent deployments
- Need high scalability

https://www.linkedin.com/in/sandeeppal/

- Require fault tolerance

### Real Enterprise Case:

A fintech company started as monolith. After reaching 2M users, they faced:

- Deployment bottlenecks
- Code merge conflicts
- Scaling issues

They split into:

- Payment Service
- Wallet Service
- User Service
- KYC Service

Result:
✔ Deployment time reduced from 2 hours → 10 minutes
✔ System uptime improved to 99.95%

---

## 🔹 3. What is Bounded Context in Microservices (DDD)?

Answer:
Bounded Context defines a logical boundary where a specific domain model applies consistently.
Each microservice should map to a bounded context.

### Example:

E-commerce domain:

| Bounded Context | Model Meaning |
|---|---|
| Order Context | Order = purchase transaction |
| Shipping Context | Order = shipment item |
| Billing Context | Order = invoice |

https://www.linkedin.com/in/sandeeppal/

Same word “Order” → different meaning per context.
This prevents:
❌ tight coupling
❌ domain confusion
❌ cross-team conflicts

---

## 🔹 4. What is API Gateway? Why critical in Microservices?

Answer:
API Gateway acts as a single entry point for all client requests.

### Responsibilities:

- Request routing
- Authentication & Authorization
- Rate limiting
- Logging & monitoring
- Request aggregation

### .NET Implementation Options:

- YARP (Microsoft)
- Ocelot

### 🔥 Real Scenario:

Mobile App requests:
GET /dashboard

Gateway internally calls:

- User Service
- Order Service
- Recommendation Service

https://www.linkedin.com/in/sandeeppal/

Aggregates response → returns single response
👉 Reduces client chattiness

---

## 🔹 5. What is Service Discovery?

Answer:
In microservices, service instances are dynamic (containers scale up/down).
Service Discovery enables services to find each other dynamically.

### Two Types:

| Type | Description |
|---|---|
| Client-side | Client queries registry |
| Server-side | Load balancer queries registry |

### Tools:

- Consul
- Eureka
- Azure Service Fabric
- Kubernetes DNS (most common modern approach)

### Example:

Order Service → needs Payment Service
It queries registry → gets available instances

---

## 🔹 6. Explain Circuit Breaker Pattern with .NET implementation.

Answer:
Circuit Breaker prevents cascading failures.

https://www.linkedin.com/in/sandeeppal/

### States:

1. Closed → normal calls
2. Open → block calls
3. Half-Open → trial calls

### .NET Implementation using Polly:

```
services.AddHttpClient("payment")
    .AddTransientHttpErrorPolicy(policy =>
        policy.CircuitBreakerAsync(3, TimeSpan.FromSeconds(30)));
```

### Real Example:

Payment service is down → without circuit breaker:

- System keeps retrying
- Thread pool exhausted
- System crashes

With circuit breaker:
✔ Stops further calls
✔ System remains stable

---

## 🔹 7. What is Saga Pattern? Explain with real transaction flow.

Answer:
Saga pattern manages distributed transactions across microservices.

### Two approaches:

1. Choreography
2. Orchestration

---

https://www.linkedin.com/in/sandeeppal/

### 🛒 Real Example: Order Placement Flow

Steps:

1. Order Service → Create Order
2. Payment Service → Charge payment
3. Inventory Service → Deduct stock
4. Shipping Service → Dispatch order

If Payment fails:

- Order Service triggers rollback
- Inventory rollback
- Order cancelled

👉 No distributed database transaction used

---

## 🔹 8. Why Database per Service pattern is critical?

Answer:
Each service must own its database to ensure:
✔ Decoupling
✔ Independent schema evolution
✔ Independent scaling
✔ Technology flexibility

### Example:

- Order Service → SQL Server
- Product Catalog → MongoDB
- Cache → Redis

If shared DB used:
❌ Tight coupling
❌ Breaking changes risk

---

https://www.linkedin.com/in/sandeeppal/

## 🔹 9. How do you maintain Data Consistency in Microservices?

Answer:
Use Eventual Consistency

### Techniques:

- Domain Events
- Event Bus
- Saga Pattern
- Outbox Pattern

### Real Example:

Order placed → publish event:
OrderCreatedEvent

Inventory Service listens → updates stock
Payment Service listens → process payment
System becomes consistent after some time

---

## 🔹 10. What is Event-Driven Architecture in Microservices?

Answer:
Services communicate via events instead of direct calls

### Benefits:

✔ Loose coupling
✔ Better scalability
✔ Resilience

### Tools:

- Azure Service Bus
- RabbitMQ

https://www.linkedin.com/in/sandeeppal/

- Kafka

### Example:

User registers → UserCreatedEvent
Email service consumes event → sends welcome email

---

## 🔹 11. What is Idempotency? Why important in Microservices?

Answer:
Idempotency ensures same request processed multiple times gives same result

### Critical in:

- Payment APIs
- Order APIs

### Example:

Client retries payment request due to timeout.
Without idempotency:
❌ User charged twice
With idempotency:
✔ Only one transaction processed

### Implementation:

Use Idempotency Key

---

## 🔹 12. What is Distributed Logging?

Answer:
In microservices, logs are spread across services.
Distributed logging aggregates logs into a central system.

### Tools:

- ELK Stack (Elastic + Logstash + Kibana)

https://www.linkedin.com/in/sandeeppal/

- Seq (popular in .NET)
- Azure Monitor

### Real Example:

User request fails → logs traced across:

- API Gateway
- Order Service
- Payment Service

---

## 🔹 13. How do you secure Microservices in .NET?

Answer:

### Security Layers:

1. Authentication
    - JWT
    - OAuth2
    - Azure AD
2. Authorization
    - Role-based
    - Policy-based
3. Transport Security
    - HTTPS
    - mTLS (service-to-service)
4. API Gateway Security

https://www.linkedin.com/in/sandeeppal/

- Token validation
- Rate limiting

---

## 🔹 14. What is CQRS? Where should you use it?

Answer:
CQRS = Command Query Responsibility Segregation
Separate:

- Write model (Commands)
- Read model (Queries)

### Example:

Write:
PlaceOrderCommand

Read:
GetOrderDetailsQuery

### Real Scenario:

E-commerce product page:

- Writes rare
- Reads heavy

So:
✔ Write → SQL
✔ Read → ElasticSearch

---

## 🔹 15. What is Domain Driven Design (DDD) in Microservices?

Answer:
DDD is an approach to design systems based on business domain and rules

https://www.linkedin.com/in/sandeeppal/

### Core Concepts:

- Entities
- Value Objects
- Aggregates
- Domain Events
- Repositories

### Why important?

Because microservices boundaries should align with business capabilities, not technical layers.

## 🔹 16. How do Microservices communicate with each other in .NET systems?

Answer:
Microservices communicate using two primary patterns:

### 1️⃣ Synchronous Communication (Request/Response)

- HTTP REST APIs
- gRPC (high performance)

✔ Immediate response
❌ Tight coupling
❌ Failure propagation risk

### 2️⃣ Asynchronous Communication (Event-based)

- Message Brokers (RabbitMQ, Azure Service Bus, Kafka)

✔ Loose coupling
✔ Resilience
✔ Eventual consistency

---

https://www.linkedin.com/in/sandeeppal/

### 🏢 Real Production Example:

Order Service → Payment Service

- First approach: HTTP call → Payment API
- Better approach: Publish OrderCreatedEvent
- Payment Service consumes → processes payment

👉 This prevents Order Service blocking due to Payment downtime

---

## 🔹 17. REST vs gRPC in Microservices — when to choose which?

Answer:

| Feature | REST | gRPC |
|---|---|---|
| Protocol | HTTP/1.1 | HTTP/2 |
| Format | JSON | Protobuf (binary) |
| Performance | Moderate | High |
| Streaming | No | Yes |
| Human readable | Yes | No |

---

### When to use REST:

✔ External APIs
✔ Public APIs
✔ Frontend integration

### When to use gRPC:

✔ Internal microservice communication
✔ High throughput systems
✔ Low latency requirements

---

### Real Example:

- Mobile app → REST

https://www.linkedin.com/in/sandeeppal/

- Microservices internal communication → gRPC

---

## 🔹 18. What is API Aggregation Pattern?

Answer:
Aggregation pattern combines multiple microservice responses into one response

### Problem:

Frontend needs data from:

- User Service
- Orders Service
- Notification Service

Without aggregation:
❌ 3 API calls from frontend
With Aggregator:
✔ 1 API call → aggregated response

---

### .NET Implementation:

- API Gateway (YARP)
- BFF (Backend For Frontend)

---

## 🔹 19. What is Backend for Frontend (BFF) pattern?

Answer:
BFF creates separate backend layer per client type

### Why?

Different clients have different needs:

| Client | Requirement |
|---|---|
| Web | Full data |
| Mobile | Lightweight |

https://www.linkedin.com/in/sandeeppal/

| | |
|---|---|
| Tablet | Medium |

---

### Real Example:

- BFF-Web
- BFF-Mobile

Each calls internal services and returns optimized response

---

## 🔹 20. What is Retry Pattern? How implemented in .NET?

Answer:
Retry pattern retries failed operations due to transient failures.

### .NET Implementation with Polly:

```
.AddTransientHttpErrorPolicy(policy =>
    policy.WaitAndRetryAsync(3, retry => TimeSpan.FromSeconds(2)));
```

---

### Real Example:

Payment API temporary failure → retry after 2 seconds
✔ Improves success rate
❌ Must be used with circuit breaker to avoid overload

---

## 🔹 21. What is Timeout Pattern?

Answer:
Timeout ensures service does not wait forever for another service.

### Example:

Order Service calls Payment Service
Timeout set: 3 seconds
If no response:
✔ fail fast
✔ return fallback response

---

https://www.linkedin.com/in/sandeeppal/

### .NET Implementation:

```
policy.TimeoutAsync(TimeSpan.FromSeconds(3));
```

---

## 🔹 22. What is Bulkhead Pattern?

Answer:
Bulkhead isolates failures by dividing system resources.

### Example:

Separate thread pools for:

- Payment calls
- Notification calls

If Notification service fails:
✔ Payment system unaffected

---

## 🔹 23. Explain Horizontal vs Vertical Scaling in Microservices.

Answer:

### Vertical Scaling:

Increase server capacity (CPU/RAM)
❌ Limited
❌ Expensive

---

### Horizontal Scaling:

Add more instances
✔ Preferred in microservices
✔ Works with containers & Kubernetes

---

### Real Example:

Order Service traffic increases
Solution:
Increase replicas from 2 → 10 pods

https://www.linkedin.com/in/sandeeppal/

---

## 🔹 24. What is Docker and its role in Microservices?

Answer:
Docker packages application + dependencies into containers

### Benefits:

✔ Same environment across dev/test/prod
✔ Fast deployment
✔ Isolation

---

### Real Example:

Each microservice runs as Docker container

---

## 🔹 25. What is Kubernetes and why used with Microservices?

Answer:
Kubernetes (K8s) is a container orchestration platform

### Responsibilities:

- Auto scaling
- Load balancing
- Self healing
- Rolling deployment
- Service discovery

---

### Real Production Scenario:

Order Service container crashes → Kubernetes automatically restarts it

---

https://www.linkedin.com/in/sandeeppal/

## 🔹 26. What is Rolling Deployment vs Blue-Green Deployment?

Answer:

### Rolling Deployment:

Gradually replaces old version
✔ No downtime
✔ Controlled rollout

---

### Blue-Green Deployment:

Two environments:

- Blue (current)
- Green (new)

Switch traffic after testing
✔ Instant rollback possible

---

## 🔹 27. What is Canary Deployment?

Answer:
Deploy new version to small percentage of users

### Example:

Release v2 to 5% users
Monitor:

- errors
- performance
- logs

If stable → rollout to 100%

---

## 🔹 28. What is Distributed Caching in Microservices?

Answer:
Distributed caching stores frequently accessed data in shared cache layer

https://www.linkedin.com/in/sandeeppal/

### Tools:

- Redis (most common in .NET)

---

### Real Example:

Product catalog service:

- DB call = 200ms
- Redis cache = 5ms

👉 huge performance gain

---

## 🔹 29. What is Cache Aside Pattern?

Answer:
Application manages cache manually.

### Flow:

1. Check cache
2. If miss → read DB
3. Store in cache
4. Return result

---

### .NET Example:

```
var product = await cache.GetAsync(id);
if(product == null)
{
    product = await db.GetProduct(id);
    await cache.SetAsync(id, product);
}
```

---

https://www.linkedin.com/in/sandeeppal/

## 🔹 30. What is Observability in Microservices?

Answer:
Observability = ability to understand system using:

### Three pillars:

1️⃣ Logs
2️⃣ Metrics
3️⃣ Traces

---

### Tools in .NET ecosystem:

- OpenTelemetry
- Azure Monitor
- Prometheus + Grafana
- Seq
- Jaeger

---

### Real Production Example:

User reports issue:
"Payment failed"
Using Observability:

- Trace request across services
- Identify failure at Payment Service
- Check logs
- Fix issue

## 🔹 31. What is YARP in .NET and when should you use it?

Answer:

https://www.linkedin.com/in/sandeeppal/

YARP (Yet Another Reverse Proxy) is a high-performance reverse proxy library by Microsoft used to build custom API Gateways in ASP.NET Core.

## Why YARP?

- Native Microsoft support
- High performance (built on ASP.NET Core pipeline)
- Fully customizable routing
- Load balancing & transformations

---

## 🏢 Real Production Use Case:

You are building a microservices-based e-commerce platform.
Instead of exposing all services:

- /orders
- /products
- /payments

You expose only:
https://api.myshop.com

YARP routes internally to appropriate services.

---

## Basic Configuration Example:

```
"ReverseProxy": {
  "Routes": {
    "orderRoute": {
      "ClusterId": "orderCluster",
      "Match": { "Path": "/orders/{**catch-all}" }
    }
  },
  "Clusters": {
    "orderCluster": {
      "Destinations": {
        "d1": { "Address": "https://orderservice" }
      }
    }
```

https://www.linkedin.com/in/sandeeppall/

```
  }
}
```

---

## 🔹 32. What is Ocelot and how is it different from YARP?

Answer:
Ocelot is a popular .NET API Gateway framework designed specifically for microservices routing.

### Key Differences:

| Feature | YARP | Ocelot |
|---|---|---|
| Owner | Microsoft | Community |
| Flexibility | High | Medium |
| Built-in features | Minimal | Many (auth, rate limiting) |
| Performance | Very high | Moderate |

---

### When to use:

- Use YARP when you want full control
- Use Ocelot for quick microservices gateway setup

---

## 🔹 33. What is Service Mesh and why needed in Microservices?

Answer:
Service Mesh manages service-to-service communication outside application code.

### Features:

- Traffic routing
- Load balancing

https://www.linkedin.com/in/sandeeppal/

- mTLS encryption
- Observability
- Retries & circuit breaking

---

### Tools:

- Istio
- Linkerd
- Consul Connect

---

### Real Scenario:

Instead of implementing retry logic in every microservice:
✔ Service Mesh handles it centrally
✔ Developers focus only on business logic

---

## 🔹 34. How do you implement Authentication in Microservices (.NET)?

Answer:
Use centralized identity provider

### Common approaches:

- OAuth2
- OpenID Connect
- Azure AD
- IdentityServer

---

https://www.linkedin.com/in/sandeeppal/

### Flow:

1. User logs in → gets JWT token
2. Client sends token in request
3. API Gateway validates token
4. Microservices trust gateway

---

### .NET Setup:

```
builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer(options =>
    {
        options.Authority = "https://identityserver";
        options.Audience = "api";
    });
```

---

## 🔹 35. What is Authorization in Microservices?

Answer:
Authorization ensures what user can access

### Types:

- Role-based (RBAC)
- Policy-based
- Claims-based

---

### Example:

```
[Authorize(Roles = "Admin")]
```

---

https://www.linkedin.com/in/sandeeppal/

### Real Scenario:

- Admin → manage products
- User → place orders

---

## 🔹 36. What is Secrets Management and how handled in .NET Microservices?

Answer:
Secrets = sensitive data:

- DB connection strings
- API keys
- Tokens

---

### Secure Storage Tools:

- Azure Key Vault
- HashiCorp Vault
- AWS Secrets Manager

---

### .NET Integration:

```
builder.Configuration.AddAzureKeyVault(...);
```

---

### Best Practices:

✔ Never store secrets in appsettings.json
✔ Use environment variables or vaults

---

https://www.linkedin.com/in/sandeeppal/

## 🔹 37. What is Message Broker in Microservices?

Answer:
Message Broker enables asynchronous communication between services.

---

### Popular Brokers:

- RabbitMQ
- Azure Service Bus
- Apache Kafka

---

### Real Example:

Order Service publishes:
OrderPlacedEvent

Payment Service consumes event and processes payment.

---

## 🔹 38. What is Outbox Pattern? Why important?

Answer:
Outbox Pattern ensures reliable event publishing

---

### Problem:

DB transaction succeeds
Event publish fails ❌
System becomes inconsistent

---

### Solution:

1. Save event in Outbox table
2. Background worker publishes events
3. Mark event as processed

---

https://www.linkedin.com/in/sandeeppal/

### Result:

✔ Guaranteed event delivery
✔ No data loss

---

## 🔹 39. What is Inbox Pattern?

Answer:
Inbox pattern ensures idempotent message processing

---

### Problem:

Message broker may deliver message twice

---

### Solution:

Store processed message IDs
If already processed → ignore

---

## 🔹 40. What is Dead Letter Queue (DLQ)?

Answer:
DLQ stores failed messages that cannot be processed.

---

### Example:

Payment service cannot process event → message moved to DLQ
Engineers can:
✔ Inspect
✔ Retry
✔ Fix issue

---

## 🔹 41. What is Event Streaming vs Messaging?

Answer:

| Feature | Messaging | Event Streaming |
|---|---|---|
| Model | Queue-based | Log-based |

https://www.linkedin.com/in/sandeeppal/

| Tool | RabbitMQ | Kafka |
|---|---|---|
| Retention | Short | Long |
| Replay | No | Yes |

---

### Use Case:

- Messaging → commands
- Streaming → analytics, event history

---

## 🔹 42. What is API Contract Testing?

Answer:
Ensures microservices follow API contract

### Types:

- Schema validation
- Integration testing

---

### Tools:

- Swagger/OpenAPI
- Pact

---

## 🔹 43. What is Consumer Driven Contract Testing?

Answer:
Consumer defines expectations from provider.
Provider must satisfy those contracts.

---

### Example:

Frontend expects:

https://www.linkedin.com/in/sandeeppal/

GET /orders → { id, amount, status }

Provider must not break contract

---

## 🔹 44. What is Anti-Corruption Layer (ACL)?

Answer:
ACL protects your microservice from external system complexity

---

### Example:

Legacy system returns messy data
ACL transforms into clean domain model before passing to service

---

## 🔹 45. What is Strangler Pattern for Microservices Migration?

Answer:
Strangler pattern is used to gradually replace monolith

---

### Steps:

1. Build new microservice
2. Route specific traffic to it
3. Slowly replace monolith features
4. Eventually remove monolith

---

### Real Enterprise Scenario:

A banking system migrated from monolith → microservices over 2 years using strangler pattern.

https://www.linkedin.com/in/sandeeppal/

## 🔹 46. What is OpenTelemetry and why is it critical in Microservices?

Answer:
OpenTelemetry is an open standard observability framework used to collect:

- Logs
- Metrics
- Distributed traces

across microservices.

---

### Why critical?

In microservices, a single request may travel across 10+ services.
Without OpenTelemetry:
❌ impossible to trace failures
With OpenTelemetry:
✔ full request tracing
✔ latency tracking
✔ dependency mapping

---

### .NET Implementation:

```
builder.Services.AddOpenTelemetry()
    .WithTracing(builder => builder.AddAspNetCoreInstrumentation()
    .AddHttpClientInstrumentation()
    .AddJaegerExporter());
```

---

### Real Scenario:

User reports:
"Payment failed"
Using OpenTelemetry:

- Trace request ID
- Identify delay in Payment Service
- Check downstream dependency failure

https://www.linkedin.com/in/sandeeppal/

---

## 🔹 47. What is Correlation ID in Microservices?

Answer:
Correlation ID is a unique request identifier passed across all services.

---

### Flow:

Client Request → API Gateway assigns ID:
X-Correlation-ID: 12345

All services propagate same ID

---

### Benefit:

✔ Trace logs across services
✔ Debug production issues faster

---

## 🔹 48. What is Distributed Tracing?

Answer:
Distributed tracing tracks complete lifecycle of a request across services.

---

### Example Flow:

API Gateway → Order Service → Payment Service → Inventory Service

Tracing shows:

- latency per service
- bottlenecks
- failure points

---

### Tools:

- Jaeger

https://www.linkedin.com/in/sandeeppal/

- Zipkin
- Azure Application Insights

---

## 🔹 49. What is Clean Architecture in Microservices?

Answer:
Clean Architecture separates system into independent layers.

### Layers:

1. Domain → Entities, business rules
2. Application → Use cases
3. Infrastructure → DB, APIs
4. API Layer → Controllers

---

### Benefits:

✔ Testable
✔ Maintainable
✔ Technology independent

---

### In Microservices:

Each microservice follows clean architecture internally

---

## 🔹 50. What is Vertical Slice Architecture?

Answer:
Vertical Slice organizes system by feature instead of layers

---

### Instead of:

Controllers/
Services/
Repositories/

https://www.linkedin.com/in/sandeeppal/

Use:
Features/
├── CreateOrder
├── CancelOrder
├── GetOrders

---

### Benefits:

✔ Feature isolation
✔ Faster development
✔ Easier maintenance

---

## 🔹 51. Repository Pattern in Microservices — should we use it?

Answer:
Repository pattern abstracts data access logic

---

### In Microservices:

✔ Good for domain-driven design
✔ Keeps business logic independent of DB

---

### But:

❗ Avoid over-abstraction
❗ Use EF Core DbContext directly for simple services

---

## 🔹 52. What is Unit of Work in Distributed Systems?

Answer:
Unit of Work manages transaction across multiple operations

---

### In Monolith:

Single DB transaction

---

https://www.linkedin.com/in/sandeeppal/

### In Microservices:

❌ Not possible across services
Use:
✔ Saga Pattern
✔ Event-driven transactions

---

## 🔹 53. What is Feature Toggle in Microservices?

Answer:
Feature Toggle allows enabling/disabling features without redeploying

---

### Example:

Enable new payment gateway for:

- 10% users only

---

### Tools:

- LaunchDarkly
- Azure App Configuration

---

## 🔹 54. What is Zero Downtime Deployment?

Answer:
Deploy new version without impacting users

---

### Techniques:

- Rolling Deployment
- Blue-Green
- Canary

---

https://www.linkedin.com/in/sandeeppal/

### Example:

Kubernetes gradually replaces pods while system remains live

---

## 🔹 55. What is Fail-Fast Principle?

Answer:
System should fail immediately instead of waiting.

---

### Example:

Payment API not responding:

- Fail in 2 seconds
- Return fallback response

✔ Prevents thread blocking
✔ Improves system stability

---

## 🔹 56. What is Graceful Degradation?

Answer:
System continues to function with reduced features when failures occur

---

### Example:

Recommendation service down:
Instead of failing entire API:
✔ Return product list without recommendations

---

## 🔹 57. What is Infrastructure as Code (IaC)?

Answer:
IaC defines infrastructure using code instead of manual setup

---

### Tools:

- Terraform

https://www.linkedin.com/in/sandeeppal/

- ARM Templates
- Bicep

---

Benefits:

✔ Version control
✔ Repeatable deployments
✔ Faster setup

---

## 🔹 58. What is Service Versioning Strategy in Microservices?

Answer:
Versioning ensures backward compatibility

---

Strategies:

- URI versioning → /api/v1/orders
- Header versioning
- Query string versioning

---

Best Practice:

✔ Avoid breaking changes
✔ Deprecate old versions gradually

---

## 🔹 59. What is Production Readiness Checklist for Microservices?

Answer:
Before going live, ensure:
✔ Health checks
✔ Logging & tracing
✔ Retry + circuit breaker

https://www.linkedin.com/in/sandeeppal/

✔ Security (JWT, HTTPS)
✔ Rate limiting
✔ Monitoring dashboards
✔ Alerting setup
✔ Backup & DR plan

---

## 🔹 60. Design a Production-Ready .NET Microservices Architecture (End-to-End)

Answer:

### 🏗️ Architecture Overview:

Client Layer
- Web / Mobile / API consumers

⬇️
API Gateway (YARP)
- Authentication
- Routing
- Rate limiting

⬇️
Microservices Layer
- Order Service (.NET + SQL Server)
- Product Service (.NET + MongoDB)
- Payment Service (.NET)
- Notification Service (.NET)

⬇️
Communication
- Sync → REST/gRPC
- Async → Azure Service Bus

⬇️
Data Layer

https://www.linkedin.com/in/sandeeppal/

- Database per service
- Redis cache

⬇
Observability
- OpenTelemetry
- Application Insights

⬇
Infrastructure
- Docker containers
- Kubernetes cluster
- Azure Cloud

---

## 🔥 Key Production Features:

✔ Circuit breaker (Polly)
✔ Retry policies
✔ Distributed tracing
✔ Central logging
✔ Secrets in Key Vault
✔ Auto scaling (K8s HPA)

---

## 🎯 Outcome:

✔ Highly scalable
✔ Fault tolerant
✔ Cloud-native
✔ Enterprise-ready system

# Senior .NET Microservices Interview Mastery

✔ 60 Advanced Questions
✔ Real Production Architecture
✔ MNC-Level Interview Ready
✔ Cloud + System Design + .NET Combined

---

https://www.linkedin.com/in/sandeeppall/

👉 Follow for more .NET, Azure, System Design & Fullstack Interview Content
👉 Comment “MICROSERVICES MASTER” to get Architecture Case Studies (Netflix/Uber style)
👉 Save this guide for your interview preparation
https://www.linkedin.com/in/sandeeppal/

https://www.linkedin.com/in/sandeeppal/